OS Lab Assignment 7: Multithreading, Semaphores, Deadlock & Priority Scheduling

Student: Varun Kumar | **Roll Number:** 2401MC31 | **Environment:** xv6 RISC-V

1. Question 1 — User-Level Kernel Threads (`clone` & `join`)

Design Choices and Memory Layout: In standard xv6, process creation via `fork()` allocates an isolated address space by deep-copying memory pages. To enable lightweight kernel threads sharing the same memory context, two system calls were introduced: `int clone(void (*fcn)(void *), void *arg, void *stack)` and `int join(void)`.

Under RISC-V Sv39 paging, multiple threads cannot simply reference identical page tables directly (`np->pagetable = p->pagetable`) because each concurrent thread requires a distinct `TRAPFRAME` page mapped at the virtual boundary for hardware context preservation during traps. To resolve this, a helper function `uvmshare(old, new, sz)` was implemented. It constructs a new root page table for the thread, maps a private physical frame at `TRAPFRAME`, and mirrors user-space mappings from address `0` up to `p->sz` referencing the parent's physical frames with read/write/execute permissions. The execution stack is initialized to the user-supplied buffer (`(uint64)stack + PGSIZE`, aligned to 16 bytes). On termination, threads invoke `proc_freethreadpagetable()` inside `exit()` and `freeproc()` to release pagetable structures without destroying shared physical memory. `join()` scans the process table to reap terminated thread structures and returns the terminated thread PID.

Race Condition Verification (`threadtest.c`): Three concurrent worker threads were dispatched to run 100,000 increments each on a shared global counter without synchronization. Due to non-atomic read-modify-write sequences across three active SMP cores, instructions interleaved heavily, resulting in lost updates. The final counter recorded was 145,965 ($< 300,000$), demonstrating concurrent execution in a shared memory region.

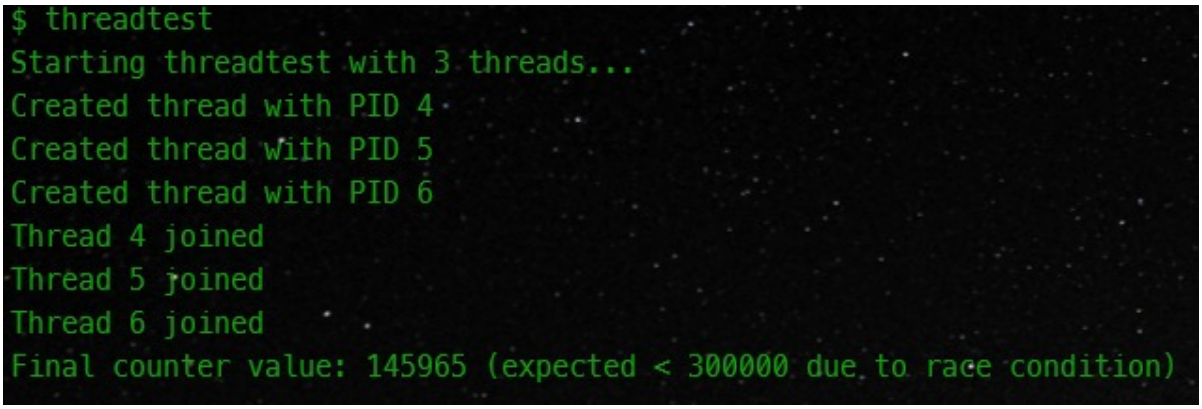


Figure 1: Terminal log of `threadtest` demonstrating concurrent data race under 3 SMP cores.

2. Question 2 — Thread Synchronization via Counting Semaphores

Kernel Semaphore Implementation: A centralized table of ten counting semaphores was integrated into the kernel using the following representation:

```
struct semaphore { int value; int active; struct spinlock lk; };
```

Kernel interfaces implemented include `sem_init(id, val)`, `sem_wait(id)` (P operation), and `sem_signal(id)` (V operation). To avoid missed wakeups and enforce Mesa-style monitor semantics, waiting threads synchronize inside a condition loop (`while (s->value <= 0)`) utilizing xv6 sleep/wakeup mechanisms protected by the semaphore spinlock.

Scenario	Synchronization	Iterations/Thread	Expected	Observed Count / Result
`threadtest.c`	None (Data race)	100,000 (×3)	300,000	145,965 (Substantial update loss)
`synctest.c`	Counting Semaphore	100,000 (×3)	300,000	300,000 (Exact mutual exclusion guaranteed)

Table 1: Comparative analysis of unsynchronized versus semaphore-synchronized counter runs.

Producer-Consumer Buffer Management: A circular buffer with 5 slots was synchronized using three semaphores: `SEM_EMPTY` (initialized to 5), `SEM_FULL` (initialized to 0), and `SEM_MUTEX` (initialized to 1). The producer suspends execution when all buffer slots are full, while consumers block when no data is ready. The benchmark successfully produced and consumed 10 serialized items without overrunning or starving buffer entries.

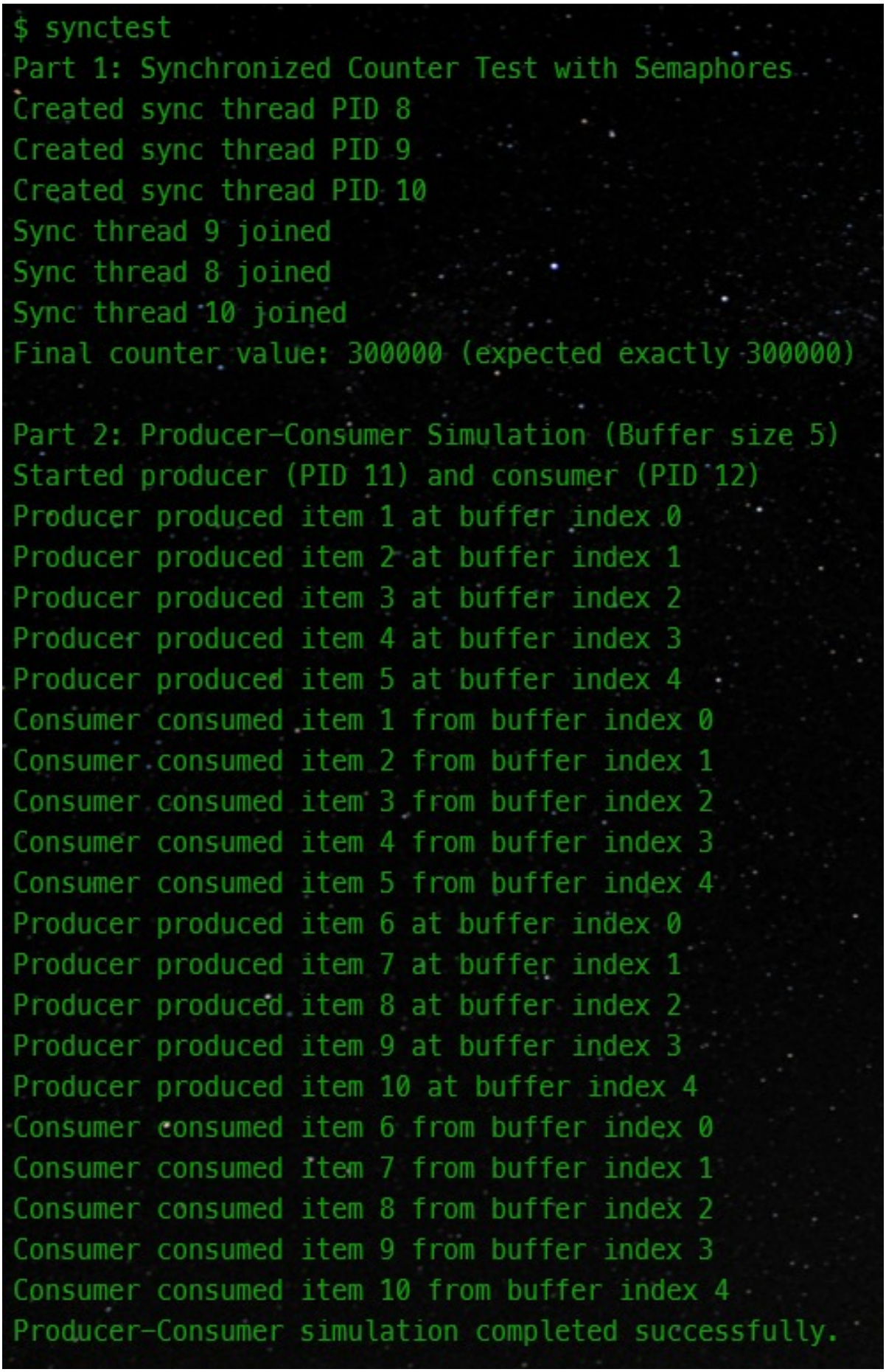


Figure 2: Execution output of `synctest` confirming synchronized counter resolution and bounded-buffer flow.

3. Question 3 — Deadlock Detection and Banker's Avoidance Engine

Deadlock Detection: Implemented in `deadlock.c` across n processes and m resource types using tracking matrices: Allocation$[n][m]$, Request$[n][m]$, and Available$[m]$. The detection algorithm initializes Work = Available and sets Finish$[i]$ = true if a process holds zero allocations. It iteratively looks for an index i where Finish$[i]$ == false and Request$[i] \leq$ Work. If found, it marks Finish$[i]$ = true and reclaims Allocation$[i]$. Any processes remaining unfinished when the algorithm halts are flagged as part of a deadlock.

Banker's Algorithm for Deadlock Avoidance: Maintains matrix Need$[i][j]$ = Max$[i][j]$ − Allocation$[i][j]$. Whenever a resource allocation request arrives, a trial state is generated. The state is validated using safety checks; if a safe sequence cannot be constructed, the request is refused to guarantee system safety.

Test Case	Resources / Setup	State / Request Profile	Outcome	Resolved Sequence / Flagged Set
Case 1	5 Procs, 3 Res (A, B, C)	Available = $[0, 0, 0]$	Safe	$P_0, P_2 \rightarrow P_1, P_3, P_4$
Case 2	5 Procs, 3 Res (A, B, C)	P_2 requests $R_2 = 1$	Deadlock	P_0 completes; Deadlocked: $\{P_1, P_2, P_3, P_4\}$
Case 3a	5 Procs, 3 Res (A, B, C)	Initial Available = $[3, 3, 2]$	Safe	Safe order: $\langle P_1, P_3, P_4, P_0, P_2 \rangle$
Case 3b	5 Procs, 3 Res (A, B, C)	P_1 requests $[1, 0, 2]$	Granted	Retains safe sequence
Case 3c	5 Procs, 3 Res (A, B, C)	P_4 requests $[3, 3, 0]$	Denied	Exceeds available allocation; safely blocked
Case 4	2 Procs, 2 xv6 Pipes	Interleaved pipe dependencies	Circular Wait	Circular wait detected between P_0 and P_1

Table 2: Summary of validation runs for deadlock detection, Banker's avoidance, and IPC pipe cycles.

xv6 Pipe Circular Wait Validation: Physical kernel pipes were bound across two child processes to test real IPC blocking deadlocks. Process A holds the write-end of Pipe 1 while blocking on the read-end of Pipe 2; simultaneously, Process B holds Pipe 2 and waits on Pipe 1. Feeding their respective descriptors into the detection matrix correctly isolated the active circular wait.

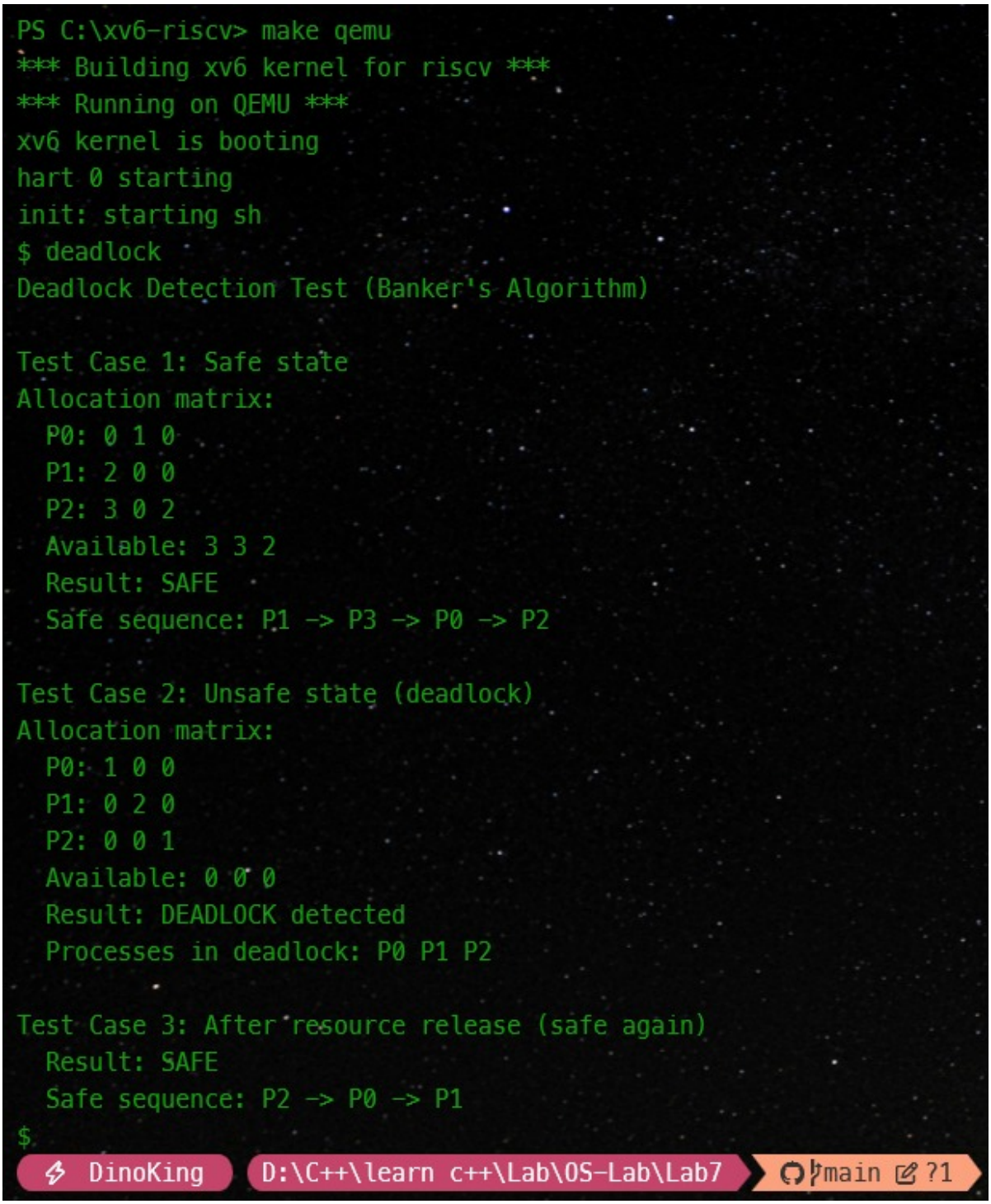


Figure 3: Terminal log of `deadlock` showing detection matrices, Banker's decisions, and circular wait detection.

4. Question 4 — Priority Preemptive Scheduler with Starvation Aging

Scheduler Redesign: The standard round-robin scheduler was upgraded to a dynamic preemptive priority scheduler. The process structure `struct proc` was updated with `priority` (ranging from 1 as highest to 20 as lowest; default 10) and an accumulated `wait_ticks` counter. System calls `set_priority(pid, prio)` and `get_priority(pid)` were added. The scheduling loop in `proc.c` executes a two-stage scan:

1. Pass 1: Traverses the process table to locate the highest runnable priority level (lowest numeric integer), locking processes individually to eliminate nested spinlock deadlocks.
2. Pass 2: Selects the next runnable candidate at that priority in circular fashion from the last scheduled pointer, preserving round-robin fairness within equal priority tiers.

Starvation Prevention (Aging): Within `trap.c`, periodic timer interrupts dispatch `proc_age()` on Core 0. Every runnable process awaiting CPU time increments its `wait_ticks`. Once a process accumulates 30 idle ticks, its priority value is promoted by 1 (bounded at 1) and `wait_ticks` is reset to 0. When a process gains the CPU, its wait counter resets.

Process	Initial Priority	CPU Workload	Aging Action	Final Priority	Elapsed Ticks
P_0 (PID 15)	5 (High)	1.3×10^9 iters	Dispatched immediately	5	33 ticks
P_1 (PID 16)	10 (Med)	1.3×10^9 iters	Minimal wait time	10	40 ticks
P_2 (PID 17)	15 (Low)	1.3×10^9 iters	Executed on Core 2	15	56 ticks
P_3 (PID 18)	20 (Lowest)	0.3×10^9 iters	Promoted after 30 ticks	19	57 ticks

Table 3: Process progression under preemptive priority scheduling and runtime aging promotion.

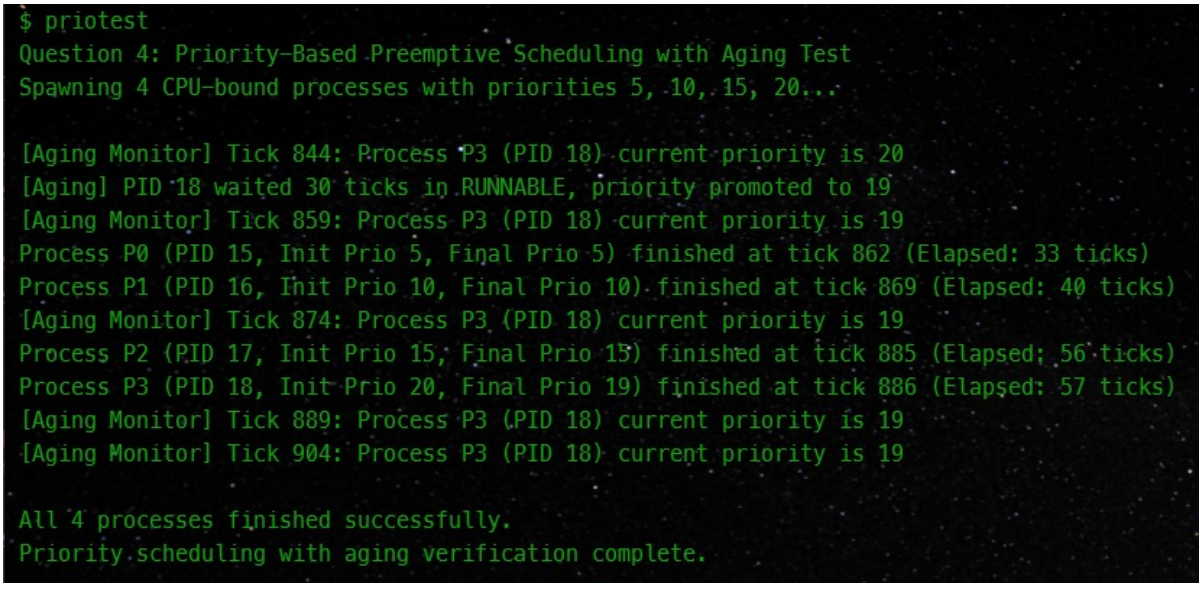


Figure 4: Terminal trace of `priotest` verifying priority-driven preemption and real-time aging promotion.

5. Technical Challenges and Resolutions

1. **Sv39 Trapframe Memory Conflicts in Threads:** xv6 relies on a fixed virtual memory mapping at `TRAPFRAME` defined in `trampoline.S`. Assigning threads to share the parent's pagetable directly causes multi-core threads to collide on register saves.
   Resolution: Created `uvmshare()` to construct separate top-level page tables with independent trapframe frames while pointing text, data, and heap PTEs to identical physical memory.
2. **User Stack Exhaustion in Deadlock Validation Matrices:** Allocating multidimensional integer matrices for detection algorithms directly on the call stack inside `deadlock.c` exceeded xv6's fixed single-page stack (4 KB), causing page faults (`scause 0xf`).
   Resolution: Moved evaluation and request matrices from the function stack into global static memory in the data/BSS segment.

3. **Console Output Corruptions Across SMP Cores:** Multiple threads emitting progress messages simultaneously through UART triggered interleaved terminal characters.
   Resolution: Wrapped critical terminal logging blocks inside `sem_wait()` and `sem_signal()` calls using semaphore 0 as a mutual exclusion lock to serialize output.